

PREGUNTAS Y RAZONAMIENTO PREVIO

LAB 6 - Sesión 6 - Deploying Machine Learning Applications

Pregunta 1 - 0.0.0.0 vs 127.0.0.1 en la red de un contenedor

Cada contenedor Docker tiene su propio stack de red aislado: una interfaz de loopback (127.0.0.1) y una interfaz virtual conectada al bridge de Docker (eth0). Arrancando Uvicorn con `--host 127.0.0.1` (o sin especificar host, valor por defecto), la aplicación solo escucha en el loopback del contenedor, ignorando eth0.

El mapeo de puertos (`-p 8000:8000`) funciona así: Docker intercepta en el host las peticiones al puerto 8000 y las reenvía a eth0 del contenedor. Si la aplicación escucha únicamente en 127.0.0.1, descarta esos paquetes y el servicio es inaccesible desde el exterior aunque el contenedor aparezca arrancado sin errores.

0.0.0.0 no es una IP concreta: indica al SO que escuche en todas las interfaces disponibles simultáneamente (loopback + eth0 + cualquier otra). Los paquetes reenviados por Docker a través del bridge llegan a eth0 y la aplicación los acepta, completando el ciclo del mapeo de puertos.

Pregunta 2 - Por qué localhost no resuelve al backend entre contenedores

En Linux, el aislamiento de red se implementa mediante network namespaces. Cada contenedor tiene su propio namespace con su tabla de routing y su loopback. Cuando el frontend llama a `http://localhost:8000`, el SO resuelve localhost como 127.0.0.1 del namespace del frontend, no del backend. El backend no existe en ese namespace.

Docker Compose crea automáticamente una red bridge compartida por todos los servicios del mismo `docker-compose.yml`. Además, el daemon Docker levanta un servidor DNS interno que registra cada nombre de servicio como entrada DNS dentro de esa red. Así, cuando el frontend resuelve 'backend', el DNS interno lo traduce a la IP asignada al contenedor backend en la red compartida. Este mecanismo es más robusto que IPs fijas, que pueden cambiar en cada reinicio.

Pregunta 3 - Optimización de la cache de capas en Docker

Docker construye imágenes de forma incremental: cada instrucción genera una capa cacheada. En cuanto detecta una capa modificada, invalida esa capa y todas las siguientes.

Si se invirtiera el orden (`COPY . .` antes de `RUN pip install`), cualquier modificación en el código -cambiar una línea de `main.py`, añadir un comentario- invalidaría la capa `COPY . .` y, en cascada, reinstalaría todas las dependencias desde cero, lo que en proyectos con FastAPI, scikit-learn y huggingface_hub puede suponer varios minutos extra por cada ciclo de desarrollo.

Con el orden correcto (`COPY requirements.txt -> pip install -> COPY . .`), la capa de instalación solo se invalida al cambiar `requirements.txt` (infrecuente). Un cambio en `main.py` únicamente invalida la última `COPY . .`, que es casi instantánea. El resultado práctico: el bucle editar-reconstruir-probar pasa de varios minutos a segundos.

Pregunta 4 - Gestión de secretos en producción

Fichero `.env` (usado en esta práctica): cómodo para desarrollo local. El riesgo principal es que el secreto persiste en disco en texto plano; un commit accidental, backup mal configurado o acceso físico a la máquina puede exponerlo. Las variables también son visibles en `docker inspect` y `ps aux`.

Variable de entorno del shell del host: el operador exporta el secreto en la terminal (`export HF_TOKEN=...`) antes de lanzar `docker compose up`. Evita ficheros en disco, pero el secreto queda en el historial del shell y en el entorno de todos los procesos hijo. Adecuada para pipelines CI/CD con gestión de variables protegidas (GitHub Actions Secrets, GitLab CI variables).

Docker Secrets (modo Swarm): almacena los secretos cifrados en el cluster y los monta como ficheros en `/run/secrets/`, nunca como variables de entorno, reduciendo la superficie de exposición. Requiere inicializar Swarm y no es compatible con `docker compose` estándar. Indicada para producción sobre Docker Swarm.

Gestores externos (AWS Secrets Manager, HashiCorp Vault): el secreto nunca sale del servicio en claro; la aplicación obtiene un token temporal y consulta el secreto vía API en tiempo de ejecución. Permiten rotación automática, auditoría granular y control de acceso por roles. Son la opción correcta para sistemas en producción a escala, entornos regulados o arquitecturas multi-servicio que exigen trazabilidad completa.

Ejercicio de Programación - Razonamiento previo (fase de diseño)

`depends_on` simple solo garantiza que el contenedor backend haya iniciado su proceso, no que la API esté operativa. Según el tiempo

de carga del modelo:

Escenario 1 - Modelo local (< 1 s): el riesgo es mínimo. El modelo se carga casi simultáneamente al arranque de Uvicorn, por lo que el frontend probablemente encuentre el servicio listo desde su primer intento de conexión.

Escenario 2 - Descarga desde Hugging Face Hub (latencia de red variable): el riesgo es alto e impredecible. El backend puede tardar varios segundos; sin health check, el frontend intentará conectar, recibirá `ConnectionRefusedError` y puede quedar en estado de error antes de que el modelo termine de descargarse.

Escenario 3 - Modelo grande con warmup (> 10 s): el fallo es seguro con `depends_on` simple. El contenedor aparecerá como 'Running' para Docker, pero la API no responderá peticiones reales durante mucho tiempo. Es imprescindible que el frontend espere a que el estado cambie a 'Healthy'.

Solución implementada: (1) HEALTHCHECK en el Dockerfile del backend que sondea `GET /health` cada 10 s con `start-period=30 s`; (2) el endpoint `/health` devuelve HTTP 200 solo cuando el modelo está cargado en memoria, y HTTP 503 mientras no lo está; (3) `depends_on` con `condition: service_healthy` en `docker-compose.yml` garantiza que el frontend no arranca hasta que el backend supera el health check.
